

California State University, Northridge

**Department of Electrical and Computer
Engineering**

ECE 425L Summer 2025

**Microcontroller Based Sun Tracking
Solar Panel**

Final Project Report

July 2nd, 2025

**Written By: Pranchi Patel and Sean
Felker**

Instructor: Prof. Shahn timerzaei

Contents:

1.0 Introduction and Problem Statement

2.0 Equipment/Components Used

3.0 Coding Style

4.0 Tasks and Results

- 4.1 Light Sensor Using Light Dependent Resistors and ADC
- 4.2 Configuration and Control Functions for the Servo
- 4.3 Attaching the Solar Panel to the Servo
- 4.4 Main Program Function

5.0 Conclusion

6.0 Source Code

- 6.1 ADC_Config.h Header File
- 6.2 ADC_Config.c Source Code File
- 6.3 delayMs.c Header File
- 6.4 delayMs.h Source Code File
- 6.5 ServoControl.h Header File
- 6.6 ServoControl.c Source Code File
- 6.7 Main.c Source Code File

1.0 Introduction and Objectives:

The goal of this project is to build a servo-controlled arm holding a solar panel that will automatically rotate the solar panel towards the side being exposed to more light. The system is controlled by the TM4C123GH6PM microcontroller. Light dependent resistors (LDR) will be used to read the light exposure for the two sides of the solar panel. The microcontroller will respond to these readings by rotating the servo so that the solar panel is exposed to the most amount of light possible.

2.0 Equipment/Components Used:

- TM4C123GH6PM microcontroller
- EduBase-V2 base board
- SG90 servo motor kit
- 60mm x 50mm solar panel - 5V, 0.3W, 60mA
- Two 330 Ω resistors
- Two light dependent resistors: 2-5k Ω 10lux resistance, 0.2M Ω dark resistance
- Cardboard, duct tape, scotch tape, glue, screwdriver, soldering kit
- 3.3V power supply

3.0 Coding Style:

The program to drive the sun tracker is divided into various functions, which are each responsible for different parts of the system. Functions are implemented to configure and control the servo and configure the analog-to-digital converter (ADC). These will be further discussed in the next section. Variables are used to keep track of the status of the servo and readings of the ADC. An infinite loop is used to process the data of the ADC and rotate the servo accordingly, which allows the sun tracker to constantly adjust towards light.

4.0 Tasks and Results:

4.1 Light Sensor Using Light Dependent Resistors and ADC

In order to compare how much light is being exposed to each side of the solar panel, a light dependent resistor (LDR) is taped to each side and connected through a circuit to an analog

to digital converter (ADC), as shown in the figure below.

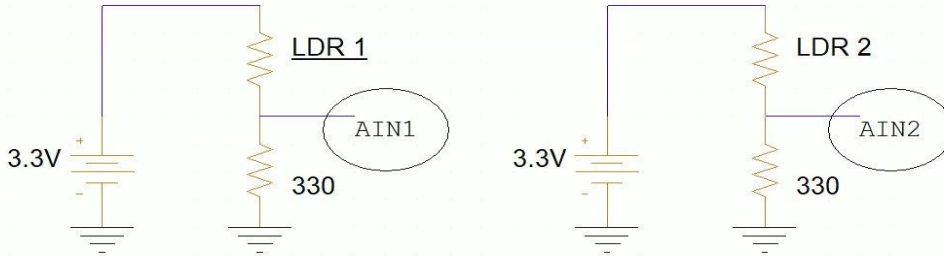


Figure 4.1: Circuit diagram connecting light dependent resistors to ADC input channels.

The defining property of an LDR is that its resistance decreases as it is exposed to more light. In our circuit, the input to the ADC is the voltage across the 330Ω resistor, which is placed in series with the LDR. The 330Ω resistor allows for a direct relationship between the voltage read by the ADC and the resistance of the LDR. The LDR's resistance decreases with more light, decreasing the voltage drop across the LDR. This increases the voltage drop across the 330Ω resistor. The opposite is also true, where less light on the LDR results in less voltage being read by the ADC.

A function is implemented to configure the analog to digital converter. GPIO ports PE1 and PE2 are configured as analog input channels AIN2 and AIN1 by setting the alternate function and analog enable registers. ADC module 0 and sample sequencer 0 is configured to read two samples, one from AIN1 and the other from AIN2. An interrupt flag is set after the second sample and conversion is completed. We used the program from lab 6 to display the reading on the LCD display to ensure that our circuit was responding correctly to light.

4.2 Configuration and Control Functions for the Servo

GPIO pin PF3's alternate function is enabled and set to output a PWM (pulse-width modulation) signal from PWM module 1, generator 3. The system clock of 50MHz is divided by 16, providing a 3.125MHz clock to the PWM module. The PWM signal is configured to count down from the load value, output low when the compare value is reached, and output high when the load value is reached. According to its datasheet, the SG90 servo is controlled by a 50Hz PWM signal, so the load value is $\frac{3.125\text{MHz}}{50\text{Hz}} = 62500$. A duty cycle of 7.5% using a compare value of 57800 is initially used for the 0° position of the servo. We can convert compare values into the resulting duty cycle using the equation $\text{duty cycle} = \left(1 - \left(\frac{\text{compare value}}{\text{load value}}\right)\right) \cdot 100\%$

To rotate the servo left and right, the compare value of the PWM signal is changed to change the duty cycle. Through testing, we found that a compare value of 54600 corresponds with the -90° position, and 61000 corresponds with the +90° position. This allowed us to roughly estimate $\frac{(61000-54600)\text{steps}}{180^\circ} \approx 35$ steps per degree of servo rotation. We used this idea to create the

`servoLeft()` and `servoRight()` functions. `servoLeft()` decreases the compare value by 35, resulting in about 1° of rotation left. `servoRight()` increases the compare value by 35, rotating the servo 1°

right. Using a conditional statement, these functions are designed to not rotate if it would result in the angle being outside of the -90° to $+90^\circ$ range. This ensures that the servo stays within its operational range so it can behave as expected.

The base board push buttons were also set up as level-triggered interrupts to rotate the servo left and right using the functions mentioned above. The user can hold down one of two buttons to manually move the solar panel to the left or right. A third button can be pressed to reset back to the 0° position.

4.3 Attaching the Solar Panel to the Servo

A cardboard arm is built, and a solar panel is glued to the top of it. The bottom is screwed to the top of the servo so that when the servo rotates, the arm will move the solar panel left and right in a circular motion.

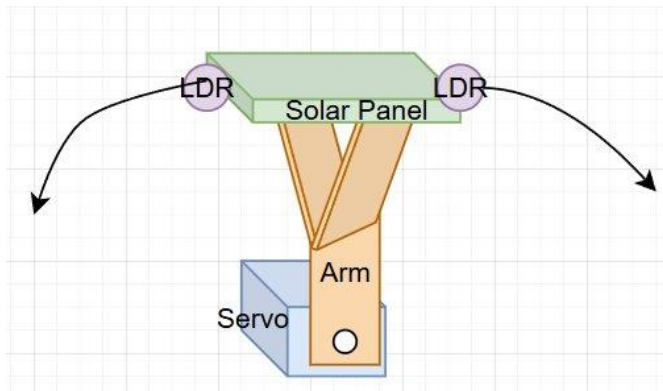


Figure 4.3: Diagram of the servo connected to the arm which holds the solar panel.

4.4 Main Program Function

Using the configurations discussed in the previous sections, the main function combines everything to drive the sun tracking solar panel. First, the ADC takes samples from both LDR circuits and stores the digital values as variables. These are converted to their analog values using the equation $V_{ANALOG} = V_{DIGITAL} \cdot \frac{V_{ref}}{2^N - 1}$, where $V_{ref} = 3.3V$ and $N = 12$ bits for our ADC. If the difference between the analog readings is more than $0.15V$, then the MCU calls one of the servo functions to move the solar panel either left or right, depending on which reading was larger. If the difference is not more than $0.15V$, then the servo is not rotated. A delay of 100 milliseconds is taken before this process is repeated again. The program was tested by shining a flashlight on each side of the solar panel and observing that the solar panel would move towards the side with more light. Covering one side of the solar panel to block light caused the solar panel to move towards the other side.

5.0: Conclusion:

The most essential aspect of this project is to calculate the exact compare value for the servo motor rotation from -90° to $+90^\circ$. Also, the additional circuit with the LDR and resistor,

and how to analyze it by using the voltage divider rule is important. By performing this project, we got a clear understanding of real-world applications for an embedded system on a small scale. From observation, if the difference between the analog reading is more than 0.15V, then the servo motor will rotate according to the individual analog reading. If the LDR1 has a higher

analog voltage, then it will rotate toward that LDR and if the LDR2 has a higher analog voltage, then it will rotate toward that LDR. When both LDR get the same amount, it will stay in the same position, which was our aim. This application is more efficient in terms of charging the solar panel than not moving the solar panel.

6.0: Source Code:

6.1 ADC_Config.h Header File

```
// Header file for configuring the analog to digital converter

#include "TM4C123GH6PM.h"

// Configures PE1 and PE2 pins as ADC pins reading one sample from each
void ADC_Config(void);
```

6.2 ADC_Config.c Source Code File

```
/* Source code to configure the analog to digital converter to read
   voltages for two light dependent resistors
   12 bit ADC with 3.3V reference voltage
*/

#include "ADC_Config.h"

void ADC_Config(void){
    SYSCTL->RCGCADC |= 1;           // Enable clock to ADC module 0
    SYSCTL->RCGCGPIO |= 0x10;       // Enable clock to GPIO port E

    // Configure PE1 and PE2 for ADC module 0
    GPIOE->AFSEL |= 0x06;           // Alternate function
    GPIOE->DEN &= ~0x06;            // Disable digital
    GPIOE->DIR &= ~0x06;            // input
    GPIOE->AMSEL |= 0x06;           // Enable analog

    ADC0->ACTSS &= ~1;              // Disable sample sequencer for ADC0
during configuration
    ADC0->EMUX &= ~0x000F;          // Enable software trigger for sample
sequencer 0
    ADC0->SSMUX0 &= ~0xFFFFFFFF;    // Clear input channel selection for SS0
    ADC0->SSMUX0 |= 0x00000021;      // AIN 2 (PE1) for first reading, AIN1 (PE2)
for second reading
    ADC0->SSCTL0 |= 0x00000600;      // End sampling and enable interrupt
```

```

after the 2nd sample
    ADC0->ACTSS |= 1;                                // Enable the sample sequencer
}

```

6.3 delayMs.h Header File

```

// delayMs header file

// Delays the system by n milliseconds for a 50MHz clock
void delayMs(int n);

```

6.4 delayMs.c Source Code File

```

// n milliseconds delay for a 50MHz system clock

#include "delayMs.h"

void delayMs(int n){
    int i, j;
    for(i = 0; i < n; i++){
        for(j = 0; j < 6265; j++){
            {} // Do nothing for 1 ms
        }
    }
}

```

6.5 ServoControl.h Header File

```

// Header File for the controlling the servo connected to Port F3 - J5 on the base
board

#include <stdbool.h>
#include "TM4C123GH6PM.h"
#include "delayMs.h"
#define DELAY_TIME 5
#define INCREMENT 35 // Increment by 35 for about 1 degree

/*Definitions for the initial compare values for the upper limit, lower limit, and
current
    for the pulse width modulation signal controlling the servo position
*/
extern const unsigned int upperLimit;
extern const unsigned int lowerLimit;
extern volatile unsigned int compareValue;
extern volatile bool format;

// Configures the push buttons SW2, SW3, and SW5 as interrupts to control the
void buttonConfig(void);

// Configures Port F3 to output a PWM signal to control the servo
void servoConfig(void);

```

```

// Configures Port B3, B2, and B1 to control the base board LEDs 3-1
void LEDConfig(void);

// Rotates the servo right by incrementing the compareValue global variable
// Increment amount determined by the INCREMENT definition
void servoRight(void);

// Rotates the servo left by decrementing the compareValue global variable
// Decrement amount determined by INCREMENT definition
void servoLeft(void);

// "Resets" the servo position by rotating it back to about 0 degrees.
void servoReset(void);

// Delays the system by n milliseconds when called
void delayMs(int n);

// Interrupt handler for push buttons. SW2 rotates the servo left, SW3 rotates right,
// and SW4 resets the position
void GPIOD_Handler(void);

```

6.6 ServoControl.c Source Code File

```

// Source code for controlling the servo connected to the PF3 PWM signal

```

```

#include "ServoControl.h"

```

```

const unsigned int upperLimit = 60950;
const unsigned int lowerLimit = 54650;
volatile unsigned int compareValue = 57800;
volatile bool format = 1;

// Port D interrupt configuration for push buttons
void buttonConfig(void){
    SYSCCTL->RCGCGPIO |= 0x08; // Enable clock for port D
    GPIOD->DIR &= ~0x0E; // Input
    GPIOD->DEN |= 0x0E; // Digital enable

    GPIOD->IS |= 0x0C; // PD3 and PD2 level triggered
    GPIOD->IS &= ~0x02; // PD1 edge triggered
    GPIOD->IBE &= ~0x0E; // Controlled by IEV register
    GPIOD->IEV |= 0x0E; // PD3 and PD2 positive level, PD1 rising edge
    GPIOD->ICR |= 0x0E; // clear prior interrupts
    GPIOD->IM |= 0x0E; // unmask interrupt

    NVIC->IPR[3] = 1<<5; // interrupt priority level is 1
    NVIC->ISER[0] |= 0x08; // Enable IRQ3

```

```
}
```

```
// Configuration for servo using PWM module 1
```

```
void servoConfig(void){
```

```
    unsigned int loadvalue = 62500;
```

```
    SYSCTL->RCGCPWM |= 0X02; // Enable the clock to PWM module 1
```

```
    SYSCTL->RCGCGPIO |= 0x20; // Enable the clock to GPIOF
```

```
    SYSCTL->RCC |= 0x00100000; // Enable clock divider for PWM
```

```
    SYSCTL->RCC &= ~0x00E0000; // Clear the current clock divider
```

```
    SYSCTL->RCC |= 0x00060000; // Set clock divide to 16: 50MHz/16 = 3.125MHz
```

```
    delayMs(1); // Wait for PWM clock to stabilize
```

```
    PWM1->_3_CTL = 0; // Disable PWM1_3 during configuration
```

```
    PWM1->_3_GENB = 0x0000080C; // Output high when load and low when match with compare value
```

```
    PWM1->_3_LOAD = loadvalue - 1; // Load value for 50 Hz
```

```
    PWM1->_3_CMPB = compareValue - 1; // Compare value for initial 7.5% duty cycle
```

```
    PWM1->_3_CTL = 1; // Enable PWM1_3, set
```

```
    counter mode to count down
```

```
    PWM1->ENABLE |= 0x80; // Enable PWM1
```

```
    GPIOF->DIR |= 0x08; // Set PF3 pin as output (Green)
```

```
    GPIOF->DEN |= 0x08; // Set PF3 pin as digital
```

```
    GPIOF->AFSEL |= 0x08; // Set PF3 as alternate function
```

```
    GPIOF->PCTL &= ~0x0000F000; // Clear PF3's current alternate function
```

```
    GPIOF->PCTL |= 0x00005000; // Set PF3 alternate function to PWM
```

```
}
```

```
// LED configuration
```

```
void LEDConfig(void){
```

```
    SYSCTL->RCGCGPIO |= 0x02;
```

```
    GPIOB->DEN |= 0x0E;
```

```
    GPIOB->DIR |= 0x0E;
```

```
}
```

```
void servoRight(void){
```

```
    if(compareValue < upperLimit){ // Limit to
```

```
+/- 90 degrees
```

```
        compareValue = compareValue + INCREMENT;
```

```
        PWM1->_3_CMPB = compareValue;
```

```
        delayMs(DELAY_TIME);
```

```
    }
```

```

}

void servoLeft(void){
    if(compareValue > lowerLimit){ // Limit to
+/- 90 degrees
        compareValue = compareValue - INCREMENT;
        PWM1->_3_CMPB = compareValue;
        delayMs(DELAY_TIME);
    }

}

void servoReset(void){
    unsigned int middle = (upperLimit+lowerLimit) / 2;
    if(compareValue > middle){
        while(compareValue != middle){
            compareValue -= 10;
            PWM1->_3_CMPB = compareValue;
            delayMs(1);
        }
    }
    else if(compareValue < middle){
        while(compareValue != middle){
            compareValue += 10;
            PWM1->_3_CMPB = compareValue;
            delayMs(1);
        }
    }
}

void GPIOD_Handler(void){

    if(GPIOD->MIS & 0x08){ // SW4 pressed
        GPIOB->DATA |= 0x08;
        servoLeft();
        GPIOD->ICR |= 0x08; // Clear the interrupt
        GPIOB->DATA &= ~0x08;
    }

    else if(GPIOD->MIS & 0x04){ // SW3 pressed
        GPIOB->DATA |= 0x04;
        servoRight();
        GPIOD->ICR |= 0x04; // Clear the interrupt flag
        GPIOB->DATA &= ~0x04;
    }
    else if(GPIOD->MIS & 0x02){ // SW2 pressed
        servoReset();
        GPIOD->ICR |= 0x02; // Clear the interrupt flag
    }
}

```

```

        else if(GPIOD->MIS & 0x01){ // SW1 pressed
            format = !format;
            delayMs(20);
            GPIOD->ICR |= 0x01;
        }
    }
}

```

6.7 Main.c Source Code File

```

// ECE 425L: Light Detector Project
//

#include <stdio.h>
#include <math.h>
#include <stdbool.h>

#include "TM4C123GH6PM.h"
#include "ServoControl.h"
#include "ADC_Config.h"
#include "delayMs.h"

int main(void){

    // Servo control functions
    buttonConfig();
    LEDConfig();
    servoConfig();

    // LCD functions
    LCD_GPIO_Config();
    LCD_init();

    // ADC
    ADC_Config();

    // Variables to hold the digital and analog LDR readings
    int LDR1_dig; int LDR2_dig;
    double LDR1_analog; double LDR2_analog;
    double diff1; double diff2;
    double diffLimit = 0.15;

    while(1){
        ADC0->PSSI |= 1; //
        Begin ADC conversion
        while((ADC0->RIS & 1) == 0){ // Wait for ADC conversion to be
        completed
        }
        // Store the sample results
        LDR1_dig = ADC0->SSFIFO0;
        LDR2_dig = ADC0->SSFIFO0;
    }
}

```

```

    ADC0->ISC = 1;
        // Clear the raw interrupt flag

    // Calculate the analog values
    LDR1_analog = (LDR1_dig * 3.3) / 4095;
    LDR2_analog = (LDR2_dig * 3.3) / 4095;

    // Determine the difference to rotate the servo
    diff1 = fabs(LDR1_analog - LDR2_analog);
    diff2 = fabs(LDR2_analog - LDR1_analog);
    if((diff1 <= diffLimit) || (diff2 <= diffLimit)){ // Difference is not
high enough -> do nothing
    }
    else if(LDR1_analog > LDR2_analog){
        servoLeft();
    }
    else if(LDR1_analog < LDR2_analog){
        servoRight();
    }

    delayMs(100);
}
return 0;
}

```